

Inigo Quilez :: articles :: wavelet image compression - 2007



Intro

In 2002 I made some experiments on image compression with wavelets. In 2007 I decided to revisit the topic and I spend few days playing around and trying to see if I could do something useful with it. In the end I had a little demo called [Inslexia](#), which made 4th (last!) position in The Ultimate Meeting demoparty in Germany.

As with many other compression techniques based in signal processing, wavelets exploit the fact that most signals to humans are mostly smooth. That means that *usually* a sample value (say, a pixel's color) is similar to the neighbouring samples (say, the color of the pixles around). However, most Fourier based compression techniques like mp3 for sound or JPG for images, and even lossless techniques like PNG, exploit this smoothness only at a very simple level. Indeed, all these techniques divide the signal (sound or image) into tiles of samples or pixels (for example, in groups of 8x8 pixels) and exploit smoothness inside that region only, not making use of the fact that perhaps samples/pixels are similar across those boundaries and at a much bigger scale (think of an almost completely blue sky).

Wavelet compression allows us to exploit smoothness at watever level it happens, be it tiny groups of 2x2 samples, be it all the pixels of a flat image at once. Therefore, the compression ratio is further improved compared to block based methods. This means that for cases where one wants extreme compression, wavelets really can shine over traditional techniques.

The fact that one needs *extreme* compression rations today in the 21st century is quite questionable, and that's perhaps the reason JPG2000 has not succeeded yet on becoming popular and will never do (ten years on the road, and nobody knows it exists). However, luckily for me the demoscene has little to do with real-world usefulness, so playing with wavelets for compression was still a usefull thing. This article is about my little quick experiment, and about few tip and tricks, some of the practical results and some sketches of the code involved in [Inslexia](#).

The Idea

The idea is to start first with a gray scale image, and do like you would proceed for a PNG image compressor: pick your buffer and group the pixels in tiles of 2x2. Now, if you only store the average color of the four pixels of each tile you are already compressing by 1:4. Good. Of course the image resolution has decreased. Let's fix it by storing the real value of the 4 pixels in a compact manner. Because these pixels are physically near to each other, we can pretty safely assume their colors will be similar to that average color that we already encoded. So, instead of storing these pixels as full gray scale values, let's store only the amount by which they are different to the average color. In fact, we only need to store three of them, cause the forth one can be deduced from the other three given the fact that we know they average. So, let's store these three values (which can be positive or negative, but probably small anyway) into a separate array, such that we have the array of corner pixels for all the image, and the three arrays of differences for the other pixels. These three arrays will compress very well cause they have small values, something that will be more true the bigger the image resolution is.

Now we don't stop here as PNG, but note that the array of average pixels actually is a new image, which has one quarter the amount of pixels of the original image - it's just a downsampled version of it. This means we can simply repeat the process to produce a new triplet of detail values and a new average image with which will be again a quarter of the size of the current image, quite like in a mipmap chain for textures. The process can be iterated until we have a one pixel by one pixel image. The beauty is that with this process we have rearranged the image data in groups or "bands" of detail differences (in groups of three) such that make sense together. Cause the difference arrays of the first iteration correspond to the highest frequency of the image (ie, that separates one pixel from the next), the second band of differences encodes the information of half that frequency, ie, the color variations that happen every four pixels, the next band works on 8 pixels frequency, and so forth. Intuitively we can see that the higher the frequency of the band, the smaller the differences will be, cause of course color one pixel appart are much more similar to each other than colors separated by 8 pixels.

So in esence, one is getting some sort of frequencial decomposition of the image but at all scales, not only at 2x2 block scale like in PNG or 8x8 like in JPG. Now, call the numbers/differences stored in each band "coefficients", and you are done with wavelet compression...

The Details

Well, not quite. Wavelets are a complex signal processing tool, and what we are doing here is nothing but scratching the very surface of the thing. In fact, what we are doing is to use one of the many possible Wavelets basis, the Haar wavelet to be more precise. But we are not going into filter-banks and dsp stuff here - instead we just will see how I implemented this simple multilevel color encoding technique and how I had my

image compressed into my demo.

We are going to replace the four pixels of a 2x2 tile, that we can call **a**, **b**, **c** and **d**, with new values **a'**, **b'**, **c'** and **d'** such that **a'** is the average of the four pixels, **b'** is the difference between **b** and the average **a'**, **c'** is the difference between **c** and the average **a'** and same for **d'**.

a	b
c	d

$$\begin{aligned} a' &= \frac{1}{4}(a + b + c + d) \\ b' &= b - a' \\ c' &= c - a' \\ d' &= d - a' \end{aligned}$$

This can be rewritten in matrix form as

$$\begin{pmatrix} a' \\ b' \\ c' \\ d' \end{pmatrix} = \frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ -1 & 3 & -1 & 1 \\ -1 & -1 & 3 & -1 \\ -1 & -1 & -1 & 3 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix}$$

We note that this matrix is not orthogonal, which means that the output vector (**a'**, **b'**, **c'** and **d'**) will have some redundant information in its coordinates. Basically **a'**, **b'**, **c'** and **d'** will be encoding redundant color information. So we orthogonalize the matrix to get

$$\begin{pmatrix} a' \\ b' \\ c' \\ d' \end{pmatrix} = \frac{1}{4} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix}$$

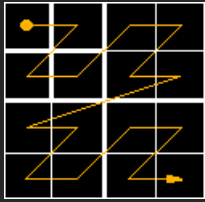
which corresponds to the following pixel operations:

$$a' = \frac{1}{4}(a + b + c + d) \quad b' = a' - \frac{b+d}{2} \quad c' = a' - \frac{c+d}{2} \quad d' = a' - \frac{b+c}{2}$$

The cool part is that this matrix equals its own inverse (up to the scale factor of 1/4), so at decodification time one can plug **a'**, **b'**, **c'** and **d'** into the matrix and get **a**, **b**, **c** and **d** back.

So the compression starts by traversing the image in groups of 2x2 pixels and computing **a'**, **b'**, **c'** and **d'**. The traversal can be done in many ways, but using the traditional raster order (top to bottom, left to right) is not the optimal. In order to explode local image coherence there are much better traversal alternatives. For example,

the one usually recommended is the Morton order, which performs recursive (fractal) zig-zags:



```
// inverse morton order (used at decompression time)
// (i,j) coordinates of the pixel, from 0 to (2^level)-1
// level : iteration level, log2(resolution)
int imorton( int i, int j, int level )
{
    int res = 0;
    for( int k=level-1; k>=0; k-- )
        res = (res<<2) + (((j>>k)&1)<<1) + (((i>>k)&1)<<0);
    return res;
}
```

```
// morton order (used at compression time)
// i: index in the sequence
// level : iteration level, log2(resolution)
// returns the pixel offset = v*resolution+u
int morton( int i, int level )
{
    int res = 0;
    for( int k=0; k < level; k++ )
    {
        res += (( i >> k ) & 1) << (k << 1);
        res += (((i>>k)&1)<<(k+level));
        i >>= 2;
    }
    return res;
}
```

Once **a'**, **b'**, **c'** and **d'** are computed, **b'**, **c'** and **d'** have to be stored in the output file/stream/datablock. In the other hand, **a'** must be kept around, such that all the **a'** of all the tiles form together a new image, which is half the size in each dimension. We now have to compress this new half-size image using the same process, repeatedly until only one we get down to an image of one single pixel, at which point the whole image has been completely encoded.

Of course, by doing all this we have not reduced the amount of data. But we have rearranged it and decorrelated it. This means that all the **b'**, **c'** and **d'** coefficients that we have been storing actually have very predictable values. For example, in the first iteration/pass, these coefficients will have really tiny values, probably in the order of ± 4 , cause images don't usually have big color variations between adjacent pixels. This means we can store the first pass coefficients with very few bits, perhaps as few as 2 or 3. Also, and here starts the fun part, you might choose to quantify these values and discard those who are small enough, quite like you would do in a JPG encoder. Now, the point of all these wavelets and recursive image processing is that unlike the JPG, here we will be discarding coefficients that are small enough **at any scale**. This is

something JPG is unable to do. So, this means, that if we have a huge patch of blue color in the sky, our wavelet compressor will be able to store it all in a couple of data values, while JPG would have to store as many values as blocks of 8x8 pixels fit in the patch.

For quantification, I found that a scaling factor proportional to $2^{(\text{level}^2)}$ works very well. Once quantification is done, a good arithmetic compressor will take proper care of outputting the data with the minimum amount of bits possible.

Implementation wise, cause you will be manipulating images of different sizes, you probably want to allocate only two buffers as big as the original image, and reuse them by ping-ponging them between levels.

```
// 512x512 gray scale image decompression
void ihaar( float **dstori, int level, const unsigned char *coeffs )
{
    const int res = 1 << level;
    const float cte = m2powf(.132f*(float)(level*level));

    // ping-pong destination/origin buffers between iterations
    float *dst = dstori[(level&1) ];
    float *ori = dstori[(level&1)^1];

    for( int i=0; i<(res*res); i++ )
    {
        const int j = i >> nivel;
        const int k = i & (res-1);
        const int m = imorton(k,j,level)*3;
        const float a = ori[i];
        const float b = cte*(float)(coeffs[m+0]-128);
        const float c = cte*(float)(coeffs[m+1]-128);
        const float d = cte*(float)(coeffs[m+2]-128);
        dst[ (j*4+0)*res + k*2+0 ] = a + b + c + d;
        dst[ (j*4+0)*res + k*2+1 ] = a - b + c - d;
        dst[ (j*4+2)*res + k*2+0 ] = a + b - c - d;
        dst[ (j*4+2)*res + k*2+1 ] = a - b - c + d;
    }

    // recurse (from 512x512 to 1x1)
    if( level<8 )
        ihaar( dstori, level+1, coeffs );
}
```

Color Images

So far we have compressed gray scale images only. For color images we are gonna use a very standard method that makes storing color very unexpensive, almost for free. The naive approach of decomposing the rgb images in three independent gray scale images is a very bad idea, you should **NEVER** do that. Instead we are going to use the popular luma/chroma decomposition, as JPG does.

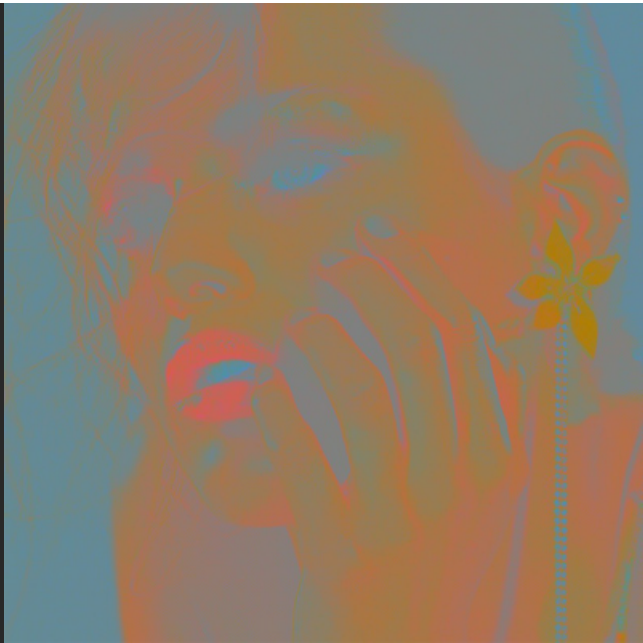
The principle on which the technique relies is that of the eye being mostly sensitive to luminance/luma values, and not that much to colors/chroma. Therefore, one can probably extract the luminance of the image (grayscale version of it) and its chroma, store the luminance at full resolution and downsample the chroma a lot yet getting a good image when upsampling the chroma and recombining it back with the original luminance.



original image (512x512 = 768 kbyte of raw data)



luminance (512x512 = 262 kbyte of raw data)



chroma (512x512 = 524 kbyte of raw data)



reconstructed image (512x512 = 264 kb of raw data)



re-upsampled luminance (32x32 = 2 kbyte of raw data)



downsampled chroma (32x32 = 2 kbyte of raw data)

In the images above I'm downsampling the chroma to 32x32 pixels image only, what is quite a lot. In fact the trick seems to behave fantastic, the reconstructed image is pretty acceptable compared to the original, but takes one third the size (as much as it's gray scale version. Here are some routines to extract the luminance **Y** and chroma **U** and **V** from the regular rgb colors and the other way around. There are many similar color conversion formuals that will do equally well, as long as they split luminance and color.

```
vec3 rgb2yuv( const vec3 & rgb )
{
    return vec3( dot(rgb,vec3(0.25f, 0.5f,0.25f) ),
                  dot(rgb,vec3(0.00f,-0.5f,0.50f) ),
                  dot(rgb,vec3(0.50f,-0.5f,0.00f) ) );
}

vec3 yuv2rgb( const vec3 & yuv )
{
    return vec3( dot(yuv, vec3(1.0f,-0.5f, 1.5f) ),
                  dot(yuv, vec3(1.0f,-0.5f,-0.5f) ),
                  dot(yuv, vec3(1.0f, 1.5f,-0.5f) ) );
}
```

Blurring the blocks

The extra compression achieved by the aggressive quantification of the detail coefficients (**b'**, **c'** and **d'**) produces blocky artifacts (unlike the JPG image compression, these blocks do not have constant size but appear in all sizes). Something that can perhaps improve the quality of the image is to blur the decompressed buffer to hide the artifacts. Of course, plain blurring will remove detail from the image. So adaptive blurring might be a better option. For **Inslexia** I decided to go for a gaussian blur which kernel radius would vary based on the variance of the rgb colors. Variance keeps low in low frequency areas, and usually increases for areas of high amount of texture. It can therefore be used as some sort of low frequency edge detector. In the following four images, the one in the top-left corner shows the decompressed image that is made of no more than 1300 bytes! The image on its right is the variance of this image. Note how the high variance areas (in white) point us to the places where we don't want to blur too much, or in other words, the areas where the radius of the gaussian blur will be small. Dark areas which are low variance areas can use a big gaussian blur kernel to perform blurring. The result is the bottom-left image. Note how much better it looks compared to the image in its right, which shows a regular blind gaussian blur applied to the original decompressed image (note the eyebrows for example).



original image (1300 bytes)



variance



variance based gaussian blur



regular gaussian blur

[inigo quilez](#) - learning computer graphics since 1994